Generating RESTful API Test Scripts with Database Constraints using OpenAPI Specification

Nuntipat Yuncharoen
Department of Computer Engineering, Faculty of Engineering, Chulalongkorn University
Thailand
ntp.ycr.acm@gmail.com

Taratip Suwannasart
Department of Computer Engineering, Faculty of Engineering, Chulalongkorn University
Thailand
taratip.s@chula.ac.th

Abstract

Nowadays, software systems are interconnected to enhance the user experience and attract new users. As such, software developers require a standard for exchanging information to streamline their work, leading to the creation of APIs. In the software development life cycle, it is essential to have documentation that outlines the software's requirements and functionality, known as a software specification. Similarly, APIs should have standardized documentation to help software developers understand their components, such as parameters, request bodies, responses, and endpoints. This is why the OpenAPI Initiative developed the OpenAPI specification. Moreover, APIs often connect with databases that store information. Relational databases have rules known as database constraints, which ensure the accuracy of stored data. When developing APIs, software testers play a crucial role, especially considering the widespread use of APIs. Software testers may have a different approach to generating test cases, potentially leading to gaps in coverage, particularly regarding database constraints. This paper proposes an approach for generating RESTful API test scripts that account for database constraints using the OpenAPI specification. This approach will be compatible with automated API testing tools, allowing software testers to achieve results more quickly and efficiently.

CCS Concepts

• **Software and its engineering**; • **Software creation and management**; • **Software verification and validation**; • **Software defect analysis**; • **Software testing and debugging**;

Keywords

RESTful API, Software Testing, OpenAPI Specification, Database Constraints, Newman, Test Scripts

ACM Reference Format:
Nuntipat Yuncharoen and Taratip Suwannasart. 2024. Generating RESTful API Test Scripts with Database Constraints using OpenAPI Specification. In *2024 The 6th World Symposium on Software Engineering (WSSE) (WSSE 2024), September 13–15, 2024, Kyoto, Japan.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3698062.3698065

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WSSE 2024, September 13–15, 2024, Kyoto, Japan
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1708-6/24/09
https://doi.org/10.1145/3698062.3698065

1 INTRODUCTION

Since 2015, the microservice architecture has become quite popular due to its independence, ease of deployment, and design based on business domains [1]. Each microservice may connect with others using RESTful APIs, which serve as a standard for information exchange. RESTful APIs are typically built according to the OpenAPI specification [2], which outlines the components of the API such as inputs, outputs, API endpoints, and other necessary features. A relational database is one of many database management systems used to store information in systems and microservices. There are many stacks of technologies that utilize relational databases, such as PERN, LAMP, and .NET Stack. To ensure the quality of a RESTful API, software testers should generate test cases to validate its functionality and identify any defects. Additionally, test cases should adhere to the OpenAPI specification. However, each software tester has different approaches to generating test cases, which depends on their experience working as software testers. The different experiences of software testers make the software testing process unpredictable in terms of time and resources. During API development, software testers may not consider the database constraints when generating test cases, which is quite important to ensure the correctness of data in the database. These databases initially have constraints such as primary key, foreign key, not null, unique, and check constraints. In API testing, test cases generated using black-box testing techniques may not be sufficient to identify all defects in a microservice, as they may not account for database constraints that improve data accuracy within the database. Moreover, testing RESTful APIs [3] can be expedited through various tools for automated testing, such as Newman, Karate, and Robot Framework. Automated RESTful API testing tools enable efficient use of automation, reducing resource usage during testing.

In this paper, the approach of generating test scripts for RESTful APIs with database constraints test cases under the single fault assumption is presented. The proposed approach will assist software testers to be a guideline in system testing in the environment of microservice-based RESTful APIs be microservice-based connecting with PostgreSQL it can reduce the gap in experience between novice and expert software testers. Moreover, this approach will decrease the time required for designing test cases and preparing test data and will be faster in obtaining test results. The input API specification is the OpenAPI Specification v3.0. So, this approach requires two inputs: the OpenAPI specification V3.0 and the PostgreSQL connection string file. It provides three outputs: test cases

with test data in CSV format, a Postman collection file in JSON format, and Newman test scripts for automated API testing. The primary objective of this approach is to assist software testers to facilitate the rapid generation of test cases and obtaining test results faster in system testing of microservice-based RESTful APIs that connect with PostgreSQL, using the OpenAPI Specification as an input.

The remainder of the paper is divided into six sections: Section 2 - Related Work, Section 3 - Background Knowledge, Section 4 - Proposed Approach, Section 5 - Experiment, Section 6 - Conclusion, and Section 7 - Future Work.

2 RELATED WORK

Sriwichainan, P. & Suwannasart, T. [4] proposed an approach for generating test scripts for web-based applications using two inputs: a URL and an XSD file. Applying boundary value testing, test cases are generated by analyzing the HTML structure and boundary values in the XSD file. Test scripts are then generated using Robot Framework and Selenium library. However, this approach does not analyze the backend API and database constraints.

Ed-douibi, H., Izquierdo, J. & Cabot, J. [5] proposed an approach for testing and generating automated test cases for REST APIs from API specifications. This approach uses the OpenAPI specification as the API model and divides test cases into two groups: nominal test cases and fault-based test cases. However, this approach does not incorporate the boundary value testing technique, which may be suitable for specification-based approach. Additionally, the test cases are not created by analyzing database constraints of database connecting APIs.

Mancas, C. [7] studied the importance of database constraints in relational databases, which consist of tables, columns, and constraints. It emphasized the significance of constraints to align data with business requirements. The study explored five database constraints: domain, not null, key/uniqueness, referential integrity, and check constraints. However, it did not propose an approach for testing these constraints, nor did it discuss the primary key constraint.

Gidvarowat, S., Suchato, A., Wanvarie, D., Pratanwanich, N. & Tuaycharoen, N. [8] proposed a case study using the Karate framework for automated API testing of an online assessment web application. The study aimed to reduce resources and found Karate more efficient than Robot framework for parallel testing. However, with only 161 test cases, it lacked automatically generated tests for database constraints. Additionally, it did not discuss Newman, another automated API testing tool.

3 BACKGROUND KNOWLEDGE

3.1 OpenAPI Specification

The OpenAPI specification is a methodology or format for describing HTTP APIs, similar to a template. It can be shared with software or system developers to facilitate adaptation with other tools. The specification not only aids in understanding the API, but also helps reduce the cost of documentation creation. The OpenAPI specification can be represented in two formats: JSON and YAML files. A sample of OpenAPI specification in YAML format is shown in Figure [9]. It is divided into 10 components, as shown in Table [10].

3.2 Database Constraints

Initially, [11] Data types are initially used to limit the data in a table. However, in some cases, data may not be accurate. For instance, a person's age should not be less than zero. Therefore, the primary role of database constraints is to control the relationships between data and ensure data accuracy, such as requiring that age is greater than or equal to zero. Using SQL, constraints can be defined for columns or tables to enforce data integrity. Additionally, if a user inputs data that violates a constraint, SQL will return an error message. The fundamental database constraints in PostgreSQL include the following:

- **Check Constraint:** Ensures that data in a column meets specified expressions**.**
- **Not null Constraint:** Ensures that data in a column must be not null.
- **Unique Constraint:** Ensures that data in a column or group of columns is unique.
- **Primary Key:** Ensures that data in a column or group of columns serves as a unique identifier for rows in the table.
- **Foreign Key:** Ensures that data in a column or group of columns matches the values appearing in a row of another table.

3.3 Robust Boundary Value Testing

Boundary value testing [12] is a well-established technique in software testing that focuses on input values within specific boundaries. The technique can be divided into two cases based on the values being tested:

- **Valid Value:** In this case, the user employs normal boundary value testing, which includes five boundary values: maximum (max), one less than the maximum (**max-1**), a nominal value (**nom**), one more than the minimum (**min+1**), and minimum (**min**).
- **Valid and Invalid Value:** In this case, the user employs robust boundary value testing, which encompasses the same five boundary values as in the normal boundary value testing: **max**, **max-1**, **nom**, **min+1**, and **min**. Additionally, two more boundary values are included: one beyond the maximum (**max+1**) and one below the minimum (**min-1**), representing invalid values. For example, if the age requirement is between 0 and 130, robust boundary value testing would encompass seven values: -1, 0, 1, 65, 129, 130, and 131.

3.4 Newman

Newman [13] is a command-line tool for testing APIs using Postman collections. It allows API testing through a collection file via the command line and can also be integrated into the software development process to ensure API quality. An example of a Newman command is shown in Table 1.

4 PROPOSED APPROACH

The proposed approach will be used as a guideline to generate test cases with test data in CSV format, a Postman collection file in JSON format, and test scripts for Newman testing tools. The approach follows these five steps, summarized briefly:

Table 1: An example of Newman command

Example command	Description
newman run bloodnow.postman_collection.json	Test all requests in collection file name "bloodnow.postman_collection.json"

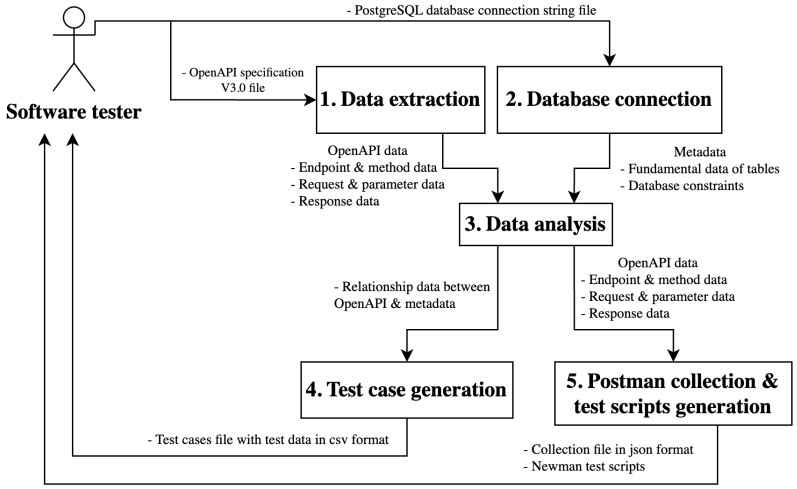


Figure 1: An overview of a proposed approach.

Data Extraction: This process will extract data from the OpenAPI specification of a RESTful API microservice that connects to a PostgreSQL relational database with database constraints.

Database Connection: This process requires a PostgreSQL database connection string file with an authorized user to obtain metadata.

Data Analysis: This process will analyze the connection between the OpenAPI data and the metadata of the PostgreSQL database to determine how inputs from the OpenAPI specification connect to fields in the database table.

Test Case Generation: This process will generate test cases, including test data in CSV format.

Postman Collection and Test Scripts Generation: Simultaneously, this process will generate a Postman collection file in JSON format and test scripts for Newman testing tools that will perform testing using the CSV test cases and the Postman collection file.

The approach is divided into five processes, as shown in Figure 1, and are explained in detail as follows:

4.1 Data Extraction

Firstly, this process will extract data from the OpenAPI specification by reading data in OpenAPI Specification v3.0. The extracted data is divided into three parts: endpoint and method data, request and parameter data, and response data, as illustrated in Figures 2, 3, and 4, respectively.

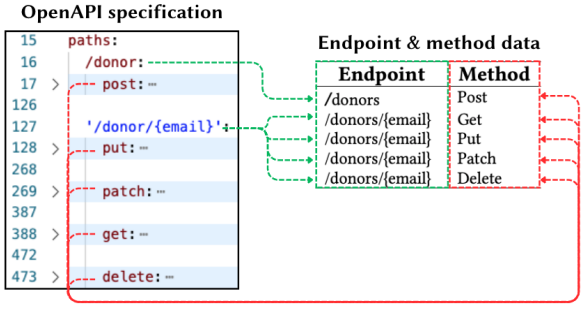
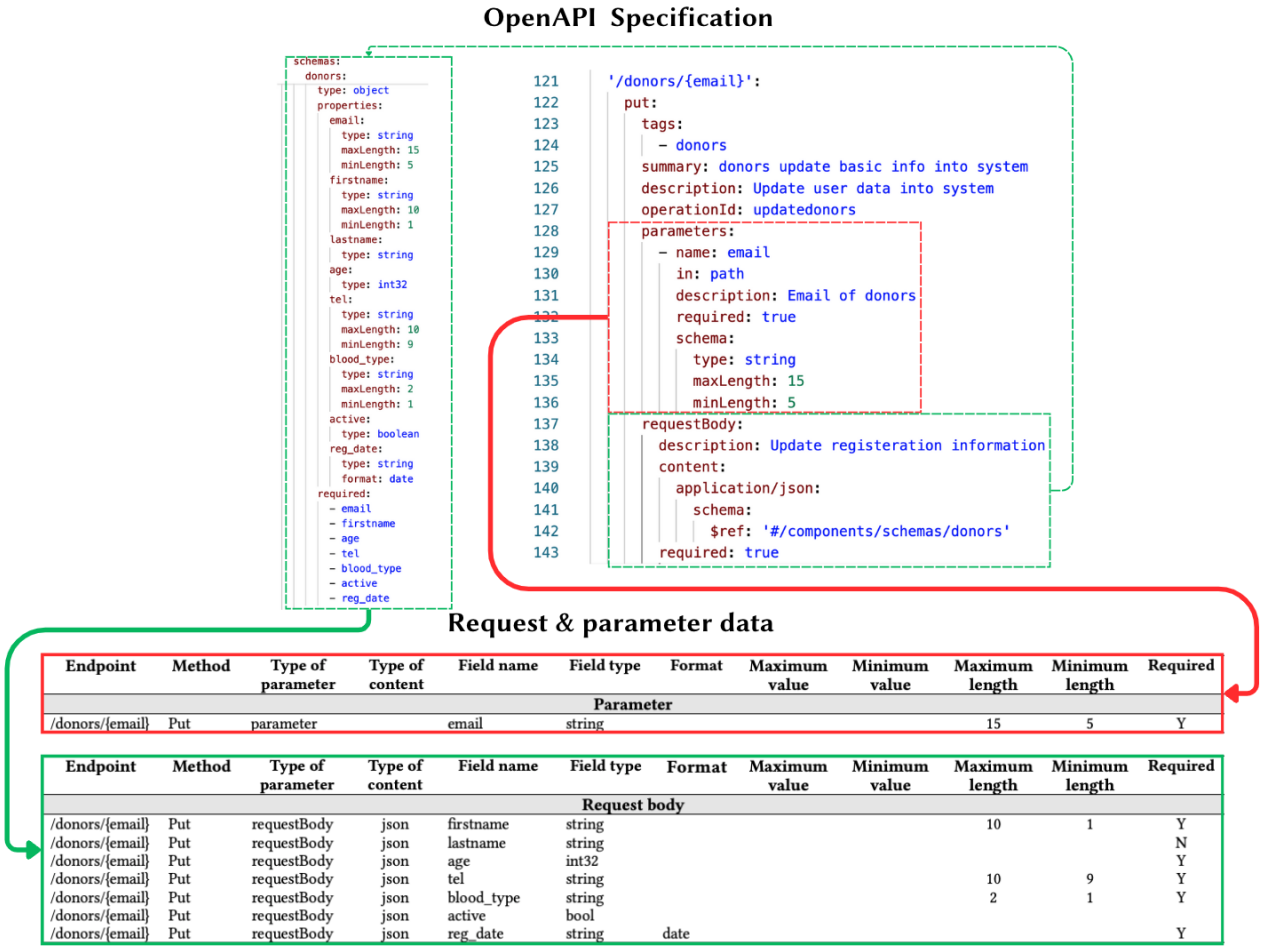


Figure 2: Extracting endpoint & method data from OpenAPI specification data.

4.2 Database Connection

Secondly, this process will receive a PostgreSQL connection string file, as shown in Figure 5, where the database name is **db_donors** with an authorized username to access the metadata of the database. After that, it will use the connection string to connect to the target database named **db_donors** to extract data from three tables: **information_schema.tables**, which describe all tables in the database; **information_schema.columns**, which describe the fields and characteristics of each field in the target table; and **information_schema.table_constraints**, which describe the database constraints of each field in a target table. The, the process will then consolidate the extracted data of three tables into a summary table, as depicted in Table 2.



Endpoint	Method	Type of parameter	Type of content	Field name	Field type	Format	Maximum value	Minimum value	Maximum length	Minimum length	Required
Parameter											
/donors/{email}	Put	parameter		email	string				15	5	Y

Endpoint	Method	Type of parameter	Type of content	Field name	Field type	Format	Maximum value	Minimum value	Maximum length	Minimum length	Required
Request body											
/donors/{email}	Put	requestBody	json	firstname	string				10	1	Y
/donors/{email}	Put	requestBody	json	lastname	string						N
/donors/{email}	Put	requestBody	json	age	int32						Y
/donors/{email}	Put	requestBody	json	tel	string				10	9	Y
/donors/{email}	Put	requestBody	json	blood_type	string				2	1	Y
/donors/{email}	Put	requestBody	json	active	bool						
/donors/{email}	Put	requestBody	json	reg_date	string	date					Y

Figure 3: Extracting request & parameter data from OpenAPI specification.

Table 2: A sample of summary of three tables from metadata

Database name	Table name	Field name	Data type	Maximum length	Primary key	Foreign key	Unique constraint	Not null constraint	Number of Check constraint
db_donors	donors	email	character varying	15	Y			Y	
		firstname	character varying	30				Y	
		lastname	character varying	6				Y	
		age	big integer						1
		tel	character varying	10			Y		
		blood_type	character varying	2		Y			
		active	boolean					Y	
		reg_date	date					Y	

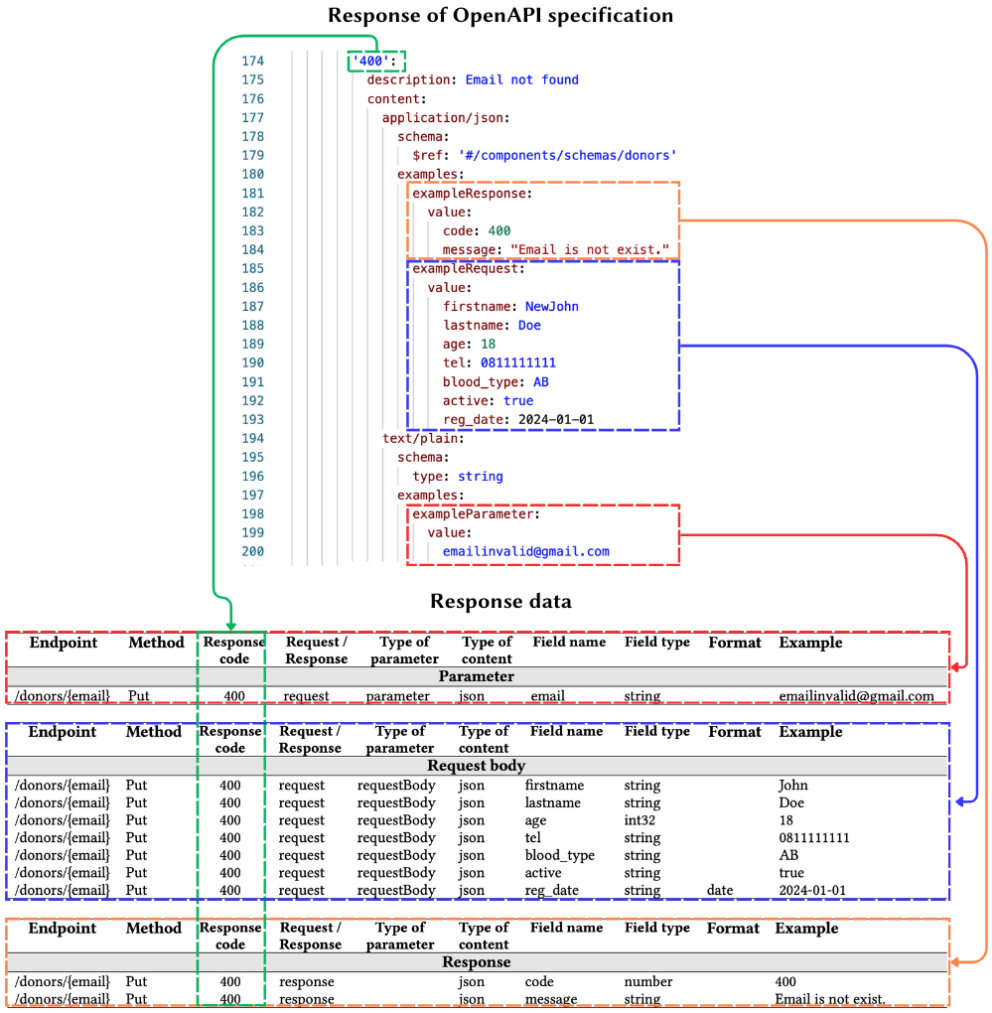


Figure 4: Extracting response data from OpenAPI specification.

```
PORT=8080
DB_URL=postgres://username:password@localhost:5432/db_donors
```

Figure 5: An example of the content in a PostgreSQL connection string file.

4.3 Data Analysis

This process establishes the connection between the OpenAPI specification data from step 4.1 and the metadata from step 4.2. Specifically, it identifies the relationships between resource names in the OpenAPI specification and tables in the metadata, as well as between request and parameter fields in the OpenAPI specification and fields in the metadata tables to find the database constraints of each field in the OpenAPI specification. The process can be divided into two parts:

4.3.1 Resource Name & Table Name: To establish the association, the process compares the names of the resource in the OpenAPI specification with the table names in the metadata. If they match, it assumes that the resource connects to that table as shown in Figure 6. that OpenAPI resource named 'donors' matches the metadata table named 'donors'.

4.3.2 Request and Parameter & Field Name in Table: Once the association between the resource name and the table is established, the process will compare the names of inputs from OpenAPI requests and parameters with the field names in the table. If they match, it is assumed that the input from the OpenAPI request and parameter is associated with the corresponding field in the table, as shown in Figure 7. In Figure 7, the field 'name' and others from OpenAPI match the field 'name' and others in the 'donors' table from metadata.

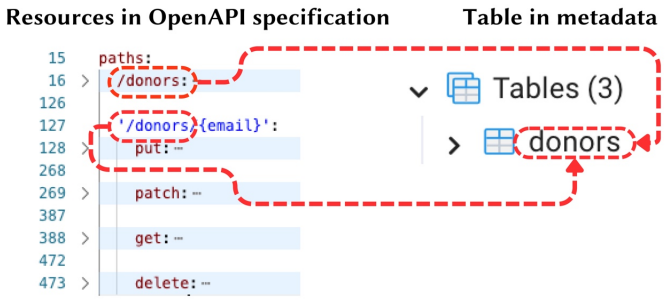


Figure 6: An example of the relationship between a resource in OpenAPI and a table in metadata.

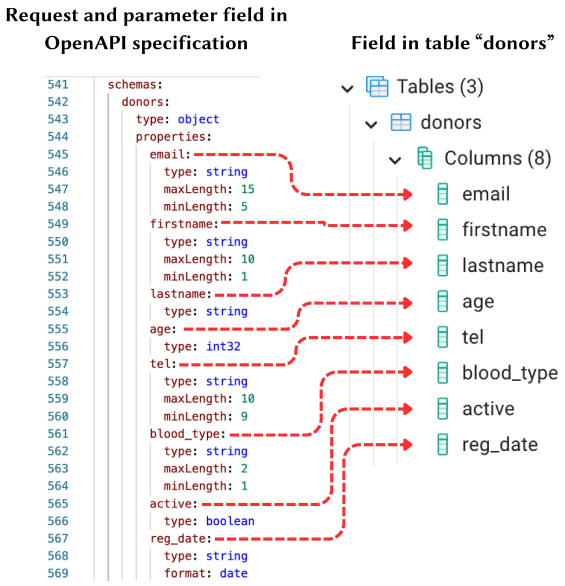


Figure 7: An example of the relationship between input fields in OpenAPI and fields in the metadata table.

Finally, the process will then summarize the results as shown in Table 3. However, if there is no match in subprocesses 3.1 or 3.2, it means that the input field of the OpenAPI specification has no database constraints.

4.4 Test Case Generation

This process will generate test cases with data from step no 3. The test cases will be generated into three types of test cases referring to the single fault assumption, and all test cases will be collected in CSV files, which are separated by endpoints and methods in the format **<Method>__<Endpoint>.CSV** such as **POST_donors.CSV**, **PUT_donors.CSV**, **PATCH_donors.CSV**, **GET_donors.CSV**, and **DELETE_donors.CSV**. All three types of test cases are sample-based test cases, specification-based test cases, and database constraint test cases.

Sample-based test case: This test case will use response data from the OpenAPI specification, as shown in Figures 8 and 9. The test cases will mirror the examples from OpenAPI to ensure that all samples in OpenAPI are tested. For a successful response code (2xx), the test case is considered valid. Conversely, a failure response code (4xx or 5xx) will indicate an invalid test case, as shown in Table 4.

Specification-based test case: This type of test case is generated by the maximum value, minimum values, maximum length or minimum length of inputs from the OpenAPI specification, metadata, and data type. When the maximum/minimum values and maximum/minimum lengths are not specified for certain inputs, the approach uses an algorithm to retrieve these values and lengths, as illustrated in Figure 10. Once the maximum/minimum values and maximum/minimum lengths are obtained as shown in Table 5, they will be used to create test data by applying the robust boundary value testing technique, as shown in Table 6. Additionally, if the input is Boolean, the process will use a randomized value testing technique, which divides the input into two cases: true and false.

Database Constraint Test Case: This test case focuses on database constraints for each field from the OpenAPI specification, examining five constraints: primary key, foreign key, unique, not null, and check constraint.

For primary key test cases (as shown in Table 7), the process checks if data exists in the table field. If data is found, it retrieves the existing data to generate test cases violating the primary key using that data. If no data exists, it generates "firstemail@t.co", inserts it into the table, and then uses the same email again to violate the primary key as shown in Table 8.

For foreign key test cases, as shown in ID no. 6 of Table 9, the process will use a random data value that does not exist in the reference table. For instance, Table 10 contains four records: A, B, O, and AB. Therefore, the test data might be a value like 'Z', as demonstrated in Table 11.

For unique constraints, this constraint will employ the same approach as primary key constraints.

For not null constraints, as shown in ID no. 1 and 2 of Table 9, the process will use a null value to violate the constraint, as demonstrated in Table 12.

For check constraint, as demonstrated in ID no. 4 of Table 9, this constraint will be tested against various expressions using six operations: greater than (>), greater than or equal to (≥), less than (<), less than or equal to (≤), equal and not equal (<> or !=). These operations will be applied in two formats: variable / operation / number (e.g., age > 15) and number / operation / number / operation

Table 3: An example of the results from process no. 3

OpenAPI specification data							Metadata			
Endpoint	**Method**	**Resource name**	**Parameter type**	**Field name**	**...**	**Required**	**Database name**	**Table name**	**Field name**	**...**
donors/{email}	Put	donors	parameter	email	...	Y	db_donors	donors	email	...
donors/{email}	Put	donors	requestBody	firstname	...	Y	db_donors	donors	firstname	...
donors/{email}	Put	donors	requestBody	lastname	...	Y	db_donors	donors	lastname	...
donors/{email}	Put	donors	requestBody	age	...	Y	db_donors	donors	age	...
donors/{email}	Put	donors	requestBody	tel	...	Y	db_donors	donors	tel	...
donors/{email}	Put	donors	requestBody	blood_type	...	Y	db_donors	donors	blood_type	...
donors/{email}	Put	donors	requestBody	active	...	Y	db_donors	donors	active	...
donors/{email}	Put	donors	requestBody	reg_date	...	Y	db_donors	donors	reg_date	...

```
'400':
  description: Email not found
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/donors'
      examples:
        exampleResponse:
          value:
            code: 400
            message: "Email is not exist."
        exampleRequest:
          value:
            firstname: NewJohn
            lastname: Doe
            age: 18
            tel: 0811111111
            blood_type: AB
            active: true
            reg_date: 2024-01-01
    text/plain:
      schema:
        type: string
      examples:
        exampleParameter:
          value:
            emailinvalid@gmail.com
```

Figure 8: An example of a response from the OpenAPI specification with a 4xx or 5xx response code.

```
'200':
  description: Successful operation
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/donors'
      examples:
        exampleRequest:
          value:
            firstname: NewJohn
            lastname: Doe
            age: 18
            tel: 0811111111
            blood_type: AB
            active: true
            reg_date: 2024-01-01
        exampleResponse:
          value:
            code: 200
            message: "firstname is updated."
    text/plain:
      schema:
        type: string
      examples:
        exampleParameter:
          value:
            john_doe@gmail.com
```

Figure 9: An example of a response from the OpenAPI specification with a 2xx response code.

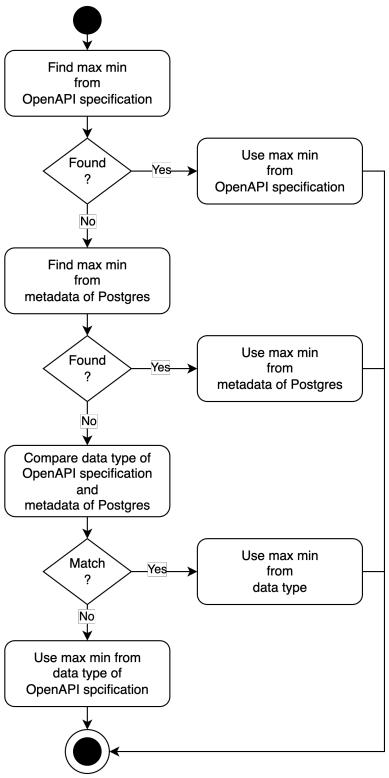


Figure 10: Activity diagram of algorithm for retrieving boundary values from multiple inputs.

TC ID	email	firstname	lastname	age	tel	blood_type	active	reg_date	Target field	Type of test data	Type of a test case
1	emailinvalid@gmail.com	NewJohn	Doe	18	0811111111	AB	true	2024-01-01		Invalid	Sample-based
2	john_doe@gmail.com	NewJohn	Doe	18	0811111111	AB	true	2024-01-01		Valid	Sample-based
3	j@g.c	val_fname1	val_lname1	10	0811111112	A	true	2024-01-01	email	Invalid	Spec-based
..	...	...	...	...	...	...	...	...	email	Valid	Spec-based
9	emailemail@e.com	val_fname1	val_lname1	10	0811111112	A	true	2024-01-01	email	Invalid	Spec-based
10	val_e1@t.co	<null>	val_lname1	10	0811111112	A	true	2024-01-01	firstname	Invalid	Spec-based
...	...	...	...	...	...	...	...	...	firstname	Valid	Spec-based
16	val_e1@t.co	aaaaaaaaaa	val_lname1	10	0811111112	A	true	2024-01-01	firstname	Invalid	Spec-based
17	val_e1@t.co	val_fname1	<null>	10	0811111112	A	true	2024-01-01	lastname	Invalid	Spec-based
...	...	...	...	...	...	...	...	...	lastname	Valid	Spec-based
23	val_e1@t.co	val_fname1	sssssss	10	0811111112	A	true	2024-01-01	lastname	Invalid	Spec-based
24	val_e1@t.co	val_fname1	val_lname1	-2147483649	0811111112	A	true	2024-01-01	age	Invalid	Spec-based
...	...	...	...	...	...	...	...	...	age	Valid	Spec-based
30	val_e1@t.co	val_fname1	val_lname1	2147483648	0811111112	A	true	2024-01-01	age	Invalid	Spec-based
31	val_e1@t.co	val_fname1	val_lname1	10	08111111	A	true	2024-01-01	tel	Invalid	Spec-based
...	...	...	...	...	...	...	...	...	tel	Valid	Spec-based
34	val_e1@t.co	val_fname1	val_lname1	10	08111111111	A	true	2024-01-01	tel	Invalid	Spec-based
35	val_e1@t.co	val_fname1	val_lname1	10	0811111112	<null>	true	2024-01-01	blood_type	Invalid	Spec-based
...	...	...	...	...	...	...	...	...	blood_type	Valid	Spec-based
39	val_e1@t.co	val_fname1	val_lname1	10	0811111112	AAA	true	2024-01-01	blood_type	Invalid	Spec-based
40	val_e1@t.co	val_fname1	val_lname1	10	0811111112	A	true	2024-01-01	active	Valid	Spec-based
41	val_e1@t.co	val_fname1	val_lname1	10	0811111112	A	false	2024-01-01	active	Valid	Spec-based

Figure 11: A sample test case of file "PUT_donors.CSV".

/ number (e.g., $5 \leq age \leq 15$). The expressions specified in Table 13 will be utilized for generating both valid and invalid test cases that violate the check constraint, as shown in Table 14.
All generated test cases of **"PUT_donors.CSV"** will be shown in Figure 11 and Figure 12.

4.5 Postman Collection and Test Scripts Generation

This process will generate test scripts for automated API testing. It will create a Postman collection file in JSON format and a Newman test script. The Postman collection file will contain all target requests defined in the OpenAPI specification, as illustrated in Figure 13. The test scripts are created by modifying the values in

Table 4: A sample of sample-based test cases

	Input								Expected result	
TC #	email	first-name	last-name	age	tel	blood_type	active	reg_date	Response code	Type of test case
1	emailinvalid@gmail.com	NewJohn	Doe	18	0811111111	AB	true	2024-01-01	400	Invalid
2	john_doe@gmail.com	NewJohn	Doe	18	0811111111	AB	true	2024-01-01	200	Valid

Table 5: Results after retrieving maximum and minimum values using the algorithm

No	End-point	Method	Field name	Maxi-mum length	Minimum length	Maxi-mum value	Minimum value	Source	Note
1	donors	Post	firstname	10	1			OpenAPI spec	
2	donors	Post	lastname	6	1			Metadata	
3	donors	Post	age			2147483647	-2147483648	Data type (OpenAPI)	Data type: int32
...	...	...	...	...	...			...	...

Table 6: Test data generated using the robust boundary value testing technique

No	firstname	lastname	age	...	Boundary value	Type of test data
1			-2147483649	...	min − 1	Invalid
2	a	s	-2147483648	...	min	Valid
3	aa	ss	-2147483647	...	min + 1	Valid
4	aaaaa	sss	10	...	norm	Valid
5	aaaaaaaaa	sssss	2147483646	...	max − 1	Valid
6	aaaaaaaaaa	ssssss	2147483647	...	max	Valid
7	aaaaaaaaaaa	sssssss	2147483648	...	max + 1	Invalid

Table 7: A sample set of data that found primary key

OpenAPI data					Metadata				
ID	Endpoint	...	Field name	...	Database name	Field name	...	Constraint	...
1	donors	...	email	...	db_donors	email	...	- Primary key	
...	...	...	...	...	...	...	...	...	

two variables: **csv_files** and **request_names** of a prepared command line template (shown in Figure 14). The **csv_files** variable contains all the test cases generated in process 4.4, while the **request_names** variable includes the names of all requests to be tested. Both the Postman collection file and the test scripts will be exported, allowing software testers to perform automated API testing by themselves.

Table 8: Sample test cases demonstrating the absence of an email in the 'donors' table

email	...	Type of test cases
firstemail@t.co	...	Valid
firstemail@t.co	...	Invalid

Table 9: A sample set of data that includes database constraints

OpenAPI data					Metadata				
ID	**Endpoint**	...	**Field name**	...	**Database name**	**Field name**	...	**Constraint**	...
1	donors/{email}	...	email	...	db_donors	email	...	- Primary key - Not null	...
2	donors/{email}	...	firstname	...	db_donors	firstname	...	- Not null	...
...	...	...	...	...	...	...	...	...	...
4	donors/{email}	...	age	...	db_donors	age	...	- Check	...
...	...	...	...	...	...	...	...	...	...
6	donors/{email}	...	blood_type	...	db_donors	blood_type	...	- Foreign key	...
...	...	...	...	...	...	...	...	...	...

Table 10: All records in the reference table named 'blood_type'

Record No	Blood_type
1	A
2	B
3	AB
4	O

Table 11: An example of invalid test data for the 'blood_type' field due to a foreign key constraint violation

Test case no	blood_type	...	Type of test cases
49	Z	...	Invalid

Table 12: Sample Not Null constraint test cases

Test case no	email	firstname	...	Type of test cases
42	<null>	val_fname1	...	Invalid
43	val_e1@t.co	<null>	...	Invalid

Table 13: An expression of check constraint collected from step 2

No	Table	Field name	Expression
1	donors	age	5 <= age <= 15

Table 14: A sample of check constraint test cases

Test case no	Expression	age	...	Type of test cases
45	5 <= age <= 15	4	...	Invalid
46	5 <= age <= 15	10	...	Valid
47	5 <= age <= 15	16	...	Invalid

TC ID	email	firstname	lastname	age	tel	blood_type	active	reg_date	Target field	Type of test data	Type of a test case
42	<null>	val_fname1	val_lname1	10	0811111112	A	true	2024-01-01	email	Invalid	DB const PK, NN
43	val_e1@t.co	<null>	val_lname1	10	0811111112	A	true	2024-01-01	firstname	Invalid	DB const NN
44	val_e1@t.co	val_fname1	<null>	10	0811111112	A	true	2024-01-01	lastname	Invalid	DB const NN
45	val_e1@t.co	val_fname1	val_lname1	4	0811111112	A	true	2024-01-01	age	Invalid	DB const CK
46	val_e1@t.co	val_fname1	val_lname1	10	0811111112	A	true	2024-01-01	age	Valid	DB const CK
47	val_e1@t.co	val_fname1	val_lname1	16	0811111112	A	true	2024-01-01	age	Invalid	DB const CK
48	val_e2@t.co	val_fname2	val_lname2	10	0811111112	A	true	2024-01-02	tel	Invalid	DB const UNQ
49	val_e1@t.co	val_fname1	val_lname1	10	0811111112	Z	true	2024-01-01	blood_type	Invalid	DB const FK
50	val_e1@t.co	val_fname1	val_lname1	10	0811111112	A	<null>	2024-01-01	active	Invalid	DB const NN
51	val_e1@t.co	val_fname1	val_lname1	10	0811111112	A	true	<null>	reg_date	Invalid	DB const NN

Figure 12: A sample test case of file "PUT_donors.CSV".

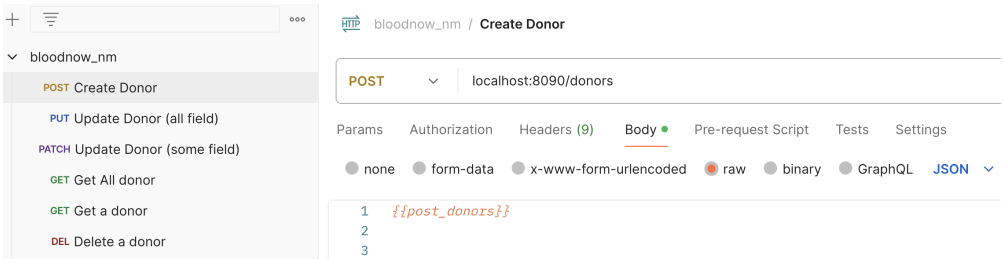


Figure 13: A sample of Postman collection file opened by Postman.

```
csv_files=("POST_donors.csv" "PUT_donors.csv" "PATCH_donors.csv" "GET_donors.csv" "DEL_donors.csv")
request_names=("Create Donor" "Update Donor (all)" "Update Donor (some)" "Get a donor" "Delete a donor")

for i in "${!csv_files[@]}"; do
 csv_file="${csv_files[i]}"
 request_name="${request_names[i]}"

 echo "Running Newman for Request: $request_name with CSV: $csv_file"

 newman run bloodnow_nm.postman_collection.json --iteration-data "$csv_file" --folder "$request_name" -r htmlextra
done
```

Figure 14: A Newman automated API test script.

Approach	Related work [4]				A proposed approach			
Technique	**RBV testing**		**Database constraints**		**RBV testing**		**Database constraints**	
The number of test cases								
Input fields	**Valid**	**Invalid**	**Valid**	**Invalid**	**Valid**	**Invalid**	**Valid**	**Invalid**
firstname	5	2	0	0	5	2	0	1
lastname	5	2	0	0	5	2	0	1
age	5	2	0	0	5	2	1	2
Total test cases	**21**				**26**			

Figure 15: result of comparing both approaches by using the amount of test cases

5 EXPERIMENT

When comparing the proposed approach with the related work [4], it is noteworthy that both methodologies address the same three fields: firstname, lastname, and age. By employing the same boundary values for all three fields, the proposed approach demonstrates its capability to generate a more comprehensive set of test cases, particularly those related to database constraints such as check and not null constraints. The proposed approach's inclusion of two database constraints results in five additional test cases compared to related work, as shown in Figure 15. The number of test cases increases due to the proposed approach's focus on testing database constraints, which are crucial for maintaining data integrity and ensuring compliance with business rules.

6 CONCLUSION

The proposed approach involves generating test cases with test data in CSV format, a Postman collection file in JSON format, and test scripts for Newman testing tools. The approach will follow these five steps: First, it will extract data from the OpenAPI specification file and metadata from the PostgreSQL connection string file. Next, it will identify the database constraints for each field from the OpenAPI specification. Then, the approach will generate test cases, including sample-based test cases, specification-based test cases, and database constraint test cases. After that, it will combine all the test cases and generate Newman test scripts and a Postman collection file. This approach can reduce the time required for generating test cases and test scripts for RESTful APIs connecting with PostgreSQL databases, using OpenAPI Specification as input. Moreover, it will serve as a guideline for developing a tool within the specified environment.

7 FUTURE WORK

In future work, the proposed approach will be used to develop a practical tool. Additional efforts could expand the approach to cover others database constraints like exclusion constraints and support various database systems, including MySQL, MongoDB, and Microsoft SQL. Another research direction involves analyzing backend code to generate test cases for identifying more defects.

References

[1] Sam Newman. 2021. Building Microservices Second edition. O'Reilly Media, Inc.,1.
[2] Joshua S. Ponelat and Lukas L. Rosenstock. 2022. Designing APIs with Swagger and OpenAPI. Manning Publications Co. 5.

[3] Dr. Anand Nayyar. 2019. Instant Approach to Software Testing. BPB Publications. 286.
[4] Pantakarn Sriwichainan and Taratip Suwannasart. 2020. Generating Test Scripts for Web-Based Applications. SSIP '20: Proceedings of the 2020 3rd International Conference on Sensors, Signal and Image Processing. Prague. Czech Republic. 84-89. https://doi.org/10.1145/3441233.3441248
[5] Hamza Ed-douibi, Javier Luis C´anovas Izquierdo and Jordi Cabot. 2018. Automatic Generation of Test Cases for REST APIs: a Specification-Based Approach. 2018 IEEE 22nd International Enterprise Distributed Object Computing Conference. Stockholm. Sweden. 181-190. https://doi.org/10.1109/EDOC.2018.00031
[6] Hamza Ed-douibi, Javier Luis C´anovas Izquierdo and Jordi Cabot. 2017. Example-driven Web API Specification Discovery. European Conference on Modelling Foundations and Applications 2017. Marburg. Germany. 267-184. http://dx.doi.org/10.1007/978-3-319-61482-3_16
[7] Christian Mancas. 2015. On the Paramount Importance of Database Constraints. Journal of Information Technology & Software Engineering Volume 5 Issue 3. 1-4. https://www.longdom.org/archive/jitse-volume-5-issue-3-year-2015.html
[8] Supagid Gidvarowart, Atiwong Suchato, Dittaya Wanvarie, Naruemon Pratanwanich and Nuengwong Tuaycharoen. 2023. Automated API Testing with Karate Framework: A Case Study of an Online Assessment Web Application. Phitsanulok. Thailand. 309-314. https://doi.org/10.1109/JCSSE58229.2023.10202050
[9] Joshua S. Ponelat and Lukas L. Rosenstock. 2022. Designing APIs with Swagger and OpenAPI. Manning Publications Co. 29.
[10] Darrel Miller, Jeremy Whitlock, Marsh Gardiner, Mike Ralphson and Ron Ratovsky. 2024. OpenAPI Object. https://spec.openapis.org/oas/v3.1.0#openapi-object
[11] The PostgreSQL Global Development Group. 2023. PostgreSQL 16.2 Documentation. The PostgreSQL Global Development Group. 62-71. https://www.postgresql.org/files/documentation/pdf/16/postgresql-16-A4.pdf
[12] Paul C. Jorgensen and Byron DeVries. 2021. Software Testing A Craftsman's Approach 5th edition. CRC Press. 79-82.
[13] Postman team. 2024. Newman the cli companion for postman. https://www.npmjs.com/package/newman#getting-started